

Distributed saga implementation

Distributed sagas can be implemented using a directed acyclic graph and defining the vertex name, request, compensating request, and status with some other fields. Then you can have logs that will help with recovery and make distributed sagas fault-tolerant and highly available. Then, you need a coordinator, as explained in the original sagas paper, which is called a **Saga Execution Coordinator (SEC)**.

The SEC is not central and stateless the 2PC coordinator. The state is managed in logs. Compensating sagas should be performed in reverse order, including all the steps. Steps can be executed in parallel. In any case, if any step is not performed it is likely that it will just not do anything and mark the compensating steps as complete. This is required to maintain consistency. This is where the commutative property comes into the picture. If the compensating saga does not know the state of any step, it will re-run the step and then compensate the transaction. For example, it will first raise a generate billing request and then cancel the billing.

[Support](#) [Sign Out](#)